Métodos clásicos y modernos para detección y localización acústica de mosquitos: un enfoque híbrido con FPGA.

Elías Álvarez
Carrera de Ing. Electrónica
Universidad Católica Nuestra Señora de la Asunción
Asunción, Paraguay
Email: elias.alvarez@universidadcatolica.edu.py

***Resumen*— Los mosquitos son vectores de enfermedades como malaria y dengue, representando un problema de salud pública de gran impacto. El monitoreo acústico surge como una alternativa de bajo costo y no invasiva para su detección y control. Este trabajo presenta una revisión histórica y comparativa de técnicas de procesamiento aplicadas a este fin. Se abordan enfoques clásicos como los phase-locked loops (PLL) y los matched filters, describiendo sus fundamentos y limitaciones. Luego, se analizan los métodos de estimación de dirección de arribo (DoA), incluyendo beamforming, MUSIC y ESPRIT, resaltando su aporte en la localización espacial. Asimismo, se discuten técnicas modernas de *deep learning*, como redes convolucionales y bayesianas, y su integración con métodos de DoA. Finalmente, se considera el uso de aceleradores basados en FPGA como solución para implementaciones en tiempo real. Este recorrido ilustra la evolución desde el DSP clásico hasta enfoques híbridos avanzados para la detección y localización de mosquitos.**

***Index Terms*—Detección de mosquitos, procesamiento de audio, dirección de arribo, *deep learning*, FPGA.**

I. Phase-Locked Loops (PLL)

Los **phase-locked loops (PLL)** son circuitos de lazo cerrado que ajustan la fase (y por ende la frecuencia) de un oscilador controlado (*VCO*/*NCO*) para que siga a la de una señal de referencia. En esencia, el PLL compara la señal de entrada con una réplica generada internamente; si detecta un desfase, corrige al oscilador hasta que ambas coincidan. Históricamente, los PLL se consolidaron en telecomunicaciones como bloques esenciales para la *recuperación de portadoras*, *demodulación de FM/PM*, *sincronización de fase/frecuencia* y *recuperación de símbolos* [2], [3], [8]. Más allá de este uso tradicional, también se aplican al seguimiento de señales periódicas en acústica y en otros dominios donde interese estimar frecuencias dominantes [11].

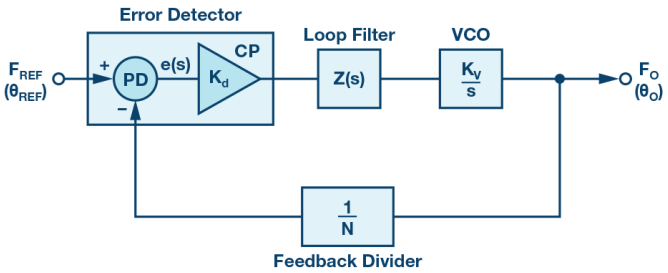


Figura 2: Diagrama en bloques de un PLL clásico de segundo orden. Se distinguen el detector de fase (PD), el filtro de lazo $Z(s)$ y el oscilador controlado K_v/s, con realimentación opcional a través de un divisor $1/N$ [4].

I-A. Estructura básica e intuición

Como muestra la Fig. 2, un PLL consta de:

- **Detector de fase (PD)**: mide el desfase entre la señal externa y la interna.
- **Filtro de lazo** $Z(s)$: convierte la señal de error en una señal de control suave.
- **Oscilador controlado (VCO/NCO)**: ajusta su frecuencia de salida según el error.
- **Divisor** $1/N$: opcional en aplicaciones de síntesis de frecuencia.

El comportamiento puede explicarse así: si la señal de entrada tiene una frecuencia ligeramente superior a la del oscilador, el detector de fase produce un error positivo que acelera al VCO. Si es más lenta, el error lo frena. El proceso se repite hasta que el VCO acompaña de forma estable a la señal externa. De este modo, el PLL actúa como un seguidor dinámico de fase y frecuencia.

I-B. Modelo lineal

Cuando el lazo ya está cerca del enganche, el análisis puede simplificarse a un modelo lineal. El error de fase $\phi_e = \phi_{\text{ref}} - \phi_{\text{vco}}$ obedece:

$$\Phi_e(s) = \frac{1}{1 + K_d K_v \frac{Z(s)}{s}} \Phi_{\text{ref}}(s), \tag{1}$$

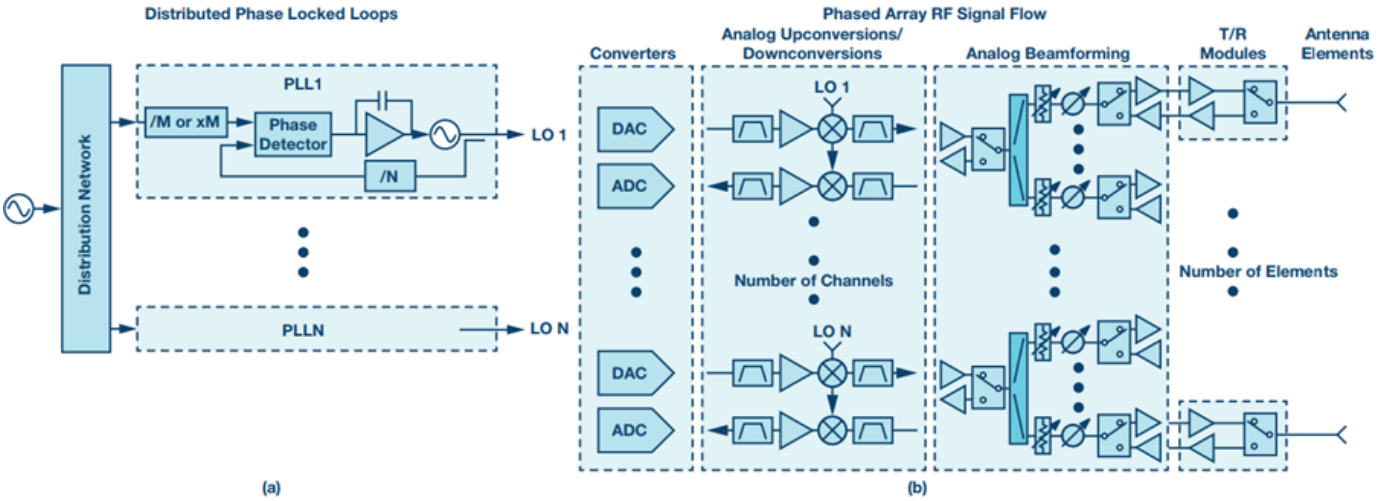


Figura 1: Ejemplo de un sistema con múltiples PLLs distribuidos en un arreglo de antenas de RF, donde cada canal genera su propio oscilador local (LO). Aunque aquí se emplea en telecomunicaciones, ilustra el concepto general de bancos de PLLs trabajando en paralelo, principio que puede aplicarse también a la detección de tonos en sistemas acústicos [7].

donde K_d es la ganancia del detector de fase, K_v la sensibilidad del VCO y $Z(s)$ el filtro.

Si el filtro es proporcional-integral (PI):

$$Z(s) = K_p + \frac{K_i}{s}, \tag{2}$$

el lazo cerrado se comporta como un sistema de segundo orden con:

$$\omega_n = \sqrt{K_d K_v K_i}, \qquad \zeta = \tfrac{1}{2}\sqrt{\tfrac{K_d K_v K_p}{K_i}},$$

donde ω_n es la frecuencia natural del lazo y ζ el factor de amortiguamiento [4], [5]. Estos parámetros controlan la rapidez con la que el PLL se engancha, su estabilidad y si aparecen sobreoscilaciones.

I-C. Rangos prácticos y ruido

Tres magnitudes prácticas caracterizan el desempeño de un PLL [4]:

- **Lock range**: rango de frecuencias donde el lazo puede mantenerse enganchado.
- **Pull-in range**: rango desde el cual el lazo es capaz de lograr enganche inicial.
- **Ancho de banda** B_L: cuánto se deja pasar de variaciones rápidas y de ruido.

Un ancho de banda grande hace al lazo rápido, pero lo expone más al ruido; uno estrecho lo protege, pero lo hace lento.

I-D. Implementación digital (DPLL)

En sistemas digitales, el VCO se reemplaza por un *NCO* (Numerically Controlled Oscillator) y todos los bloques se discretizan. El detector de fase puede implementarse, por ejemplo, mezclando la señal de entrada con una referencia en cuadratura (I y Q) y calculando:

$$e_\phi[n] = \operatorname{atan2}(Q[n], I[n]).$$

El filtro PI en forma discreta acumula el error de fase:

$$u[n] = u[n-1] + K_p\, e_\phi[n] + K_i \sum_{k=0}^{n} e_\phi[k], \tag{3}$$

$$\phi_{\text{nco}}[n+1] = \phi_{\text{nco}}[n] + K_v\, u[n]. \tag{4}$$

Aquí:

- K_p define la respuesta rápida a cambios instantáneos,
- K_i corrige errores de fase acumulados a largo plazo,
- K_v traduce la señal de control en cambio de frecuencia del NCO.

En la práctica, este cálculo se implementa con sumadores, multiplicadores y acumuladores digitales, por lo que es especialmente amigable a hardware como FPGAs. Técnicas como Tustin o bilineal se usan para discretizar K_p, K_i a partir del diseño analógico.

I-E. Banco de PLLs

Un ejemplo de este enfoque se observa en la Fig. 1. En lugar de un único lazo, puede emplearse un banco de PLLs en paralelo, cada uno sintonizado a una frecuencia f_k. Cada PLL entrega tres señales clave:

- Un indicador de enganche (lock).
- La estimación de fase/frecuencia en su canal.
- La potencia coherente asociada a esa frecuencia.

Si un conjunto de PLLs (ej. los que cubren $f_0, 2f_0, 3f_0$) se enganchan al mismo tiempo, se infiere la presencia de una señal con ese patrón armónico. Esto equivale a un análisis espectral adaptativo con muy baja latencia y es factible en hardware embebido [11].

I-F. Ventajas y limitaciones

Ventajas: bajo costo computacional, buena selectividad, fácil de implementar en FPGA/DSP. **Limitaciones**: rango de captura

limitado, no separa dos fuentes con la misma frecuencia, y puede perder enganche en ambientes con multitrayectoria severa [4].

II. Matched Filters

El **filtro adaptado** o *matched filter* es un sistema lineal que maximiza la relación señal–ruido (SNR) en la detección de una señal conocida inmersa en ruido. La idea central es simple: si se conoce la forma de la señal $s(t)$, el filtro más efectivo es aquel cuya respuesta al impulso está adaptada a ella [14], [2], [3].

II-A. Convolución y correlación

En procesamiento de señales, la **convolución** describe la salida $y(t)$ de un sistema lineal:

$$y(t) = \int r(\tau)\, h(t-\tau)\, d\tau, \tag{5}$$

donde $r(t)$ es la entrada y $h(t)$ la respuesta al impulso.

Por otro lado, la **correlación** mide la similitud entre una señal recibida $r(t)$ y una señal de referencia $s(t)$:

$$c(t) = \int r(\tau)\, s^*(\tau - t)\, d\tau. \tag{6}$$

La conexión clave es que un filtro adaptado se diseña de modo que su salida de convolución con $r(t)$ sea equivalente a la correlación entre $r(t)$ y $s(t)$ [14].

II-B. Por qué invertir la señal

Para lograr esta equivalencia, el filtro se define con una respuesta al impulso:

$$h(t) = s^*(T - t), \tag{7}$$

donde T es la duración de la señal. La inversión temporal $s(T-t)$ asegura que:

1. La salida presente un máximo bien definido justo cuando la señal $s(t)$ se alinea con $r(t)$.
2. El filtrado se pueda implementar de manera causal mediante el corrimiento apropiado.

De esta manera, la salida del filtro alcanza un pico en el instante en que la señal buscada aparece en la observación.

II-C. Definición formal

Si la señal recibida es $r(t) = s(t)+n(t)$, con $n(t)$ ruido blanco gaussiano, puede demostrarse que el filtro $h(t) = s^*(T-t)$ es el que **maximiza la SNR de salida** [14].

II-D. Implementación digital

En tiempo discreto, el matched filter se implementa como un **filtro FIR** (*Finite Impulse Response*, respuesta al impulso finita). La salida en el instante k depende de una combinación lineal de un número finito de muestras pasadas:

$$y[k] = \sum_{n=0}^{N-1} h[n]\, r[k-n], \tag{8}$$

donde $h[n]$ son los coeficientes del filtro y N su longitud.

En el caso del matched filter, los coeficientes son la versión invertida y conjugada de la señal patrón:

$$h[n] = s^*[N-1-n], \qquad n = 0, \ldots, N-1, \tag{9}$$

con $s[n]$ la secuencia conocida que se desea detectar.

Interpretación práctica:

- Cada coeficiente $h[n]$ almacena la forma de la señal buscada.
- El cálculo $y[k]$ equivale a "superponer" $s[n]$ sobre la ventana de datos más reciente de $r[n]$.
- Si las dos secuencias coinciden (salvo por ruido), la suma ponderada es máxima y $y[k]$ presenta un pico.

Ventajas de la implementación FIR:

- **Estabilidad**: al no tener retroalimentación, un FIR no puede divergir.
- **Flexibilidad**: basta con cambiar los coeficientes $h[n]$ para adaptarse a otra señal patrón.
- **Eficiencia**: la operación es solo un producto punto, fácil de vectorizar o implementar en hardware (DSP/FPGA).

En la práctica, algoritmos como la *FFT-based convolution* permiten realizar la correlación de manera aún más eficiente cuando N es grande.

II-E. Ejemplos y aplicaciones

- **Radar**: detección de ecos débiles con forma de pulso conocida.
- **Sonar**: localización de objetos bajo el agua mediante pulsos transmitidos.
- **Sismología**: identificación de terremotos pequeños correlando registros con formas de onda conocidas.
- **Bioacústica**: detección de patrones repetitivos (como zumbidos) en ambientes ruidosos.

II-F. Limitaciones

- Detecta eficazmente solo la señal exacta para la que fue diseñado.
- Variaciones de frecuencia o fase reducen su desempeño.
- Poco flexible en entornos con señales muy diversas o distorsionadas.

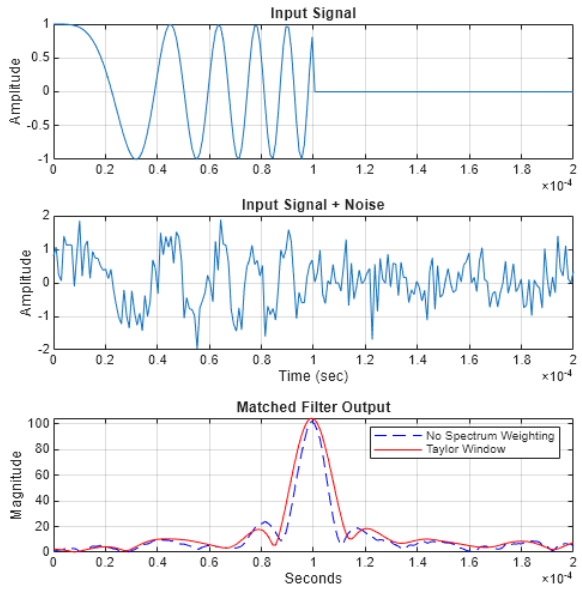


Figura 3: Ejemplo de filtrado adaptado (*matched filtering*). Arriba se muestra una señal lineal-FM (chirp) contaminada con ruido, y abajo la salida del filtro adaptado. Se observa cómo el filtro maximiza la SNR al alinear la señal con su versión invertida y conjugada, produciendo un pico bien definido en la detección. La curva azul corresponde al filtrado sin ponderación espectral y la roja al uso de una ventana de Taylor para reducir lóbulos secundarios [14].

III. Estimación de Dirección de Arribo (DOA)

La **estimación de dirección de arribo (Direction of Arrival, DOA)** busca determinar desde qué ángulo llega una onda a un arreglo de sensores. Un ejemplo ilustrativo es la ecolocación de los murciélagos: pequeñas diferencias de tiempo y fase entre ambos oídos permiten estimar la dirección del sonido (véase Fig. 5). En un arreglo de micrófonos ocurre lo mismo: cada sensor recibe la misma onda con un retardo relativo, y ese desfase contiene la información angular (Fig. 4) [15].

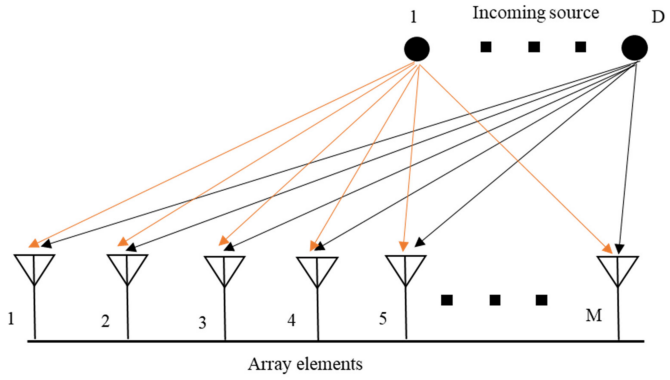


Figura 4: Arreglo de sensores lineal para estimación de DOA.

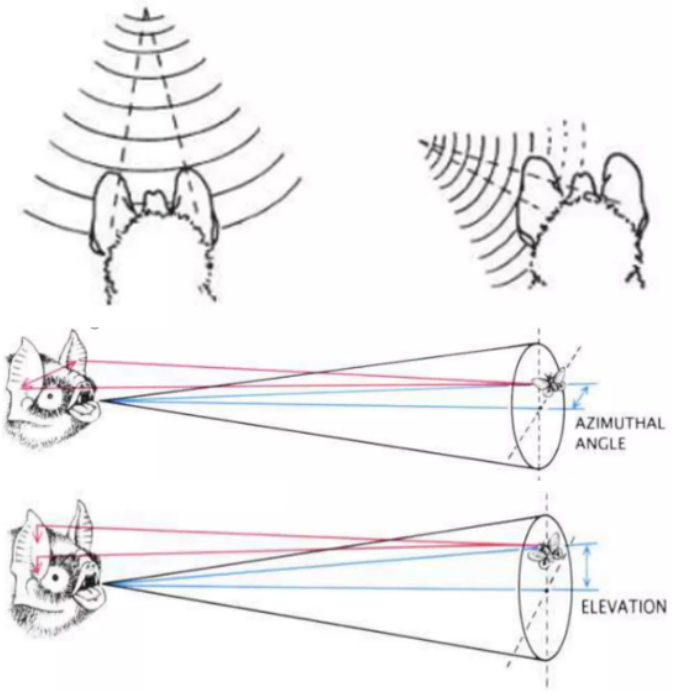


Figura 5: Ejemplo biológico de estimación de dirección de arribo en murciélagos.

III-A. *Modelo de onda plana unidimensional*

Consideremos primero una onda plana armónica propagándose a lo largo de un eje x. La señal recibida en la posición x es:

$$s(x,t) = A\, e^{j(\beta x - \omega t + \phi_0)}, \tag{10}$$

donde A es la amplitud, $\beta = 2\pi/\lambda$ el número de onda, $\omega = 2\pi f$ la frecuencia angular y ϕ_0 la fase inicial.

Esta expresión refleja dos aspectos: (i) el término temporal $e^{-j\omega t}$ describe la oscilación sinusoidal, (ii) el término espacial $e^{j\beta x}$ indica cómo cambia la fase al desplazarse en x. Cuando la onda avanza una longitud de onda λ, el exponente completa un ciclo de 2π, ilustrando su periodicidad (Fig. 6).

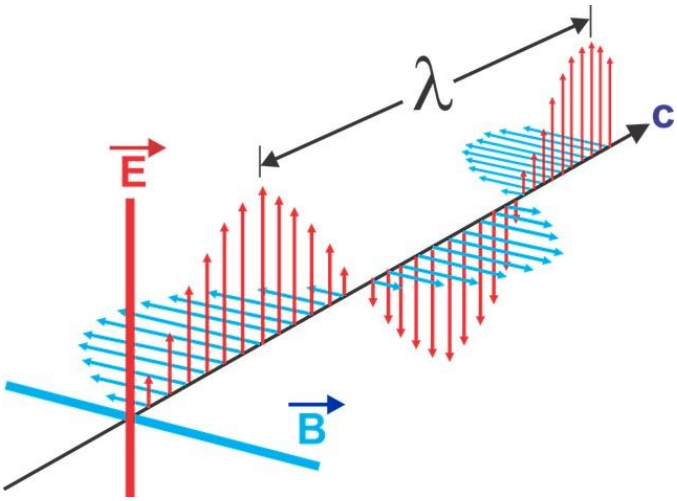


Figura 6: Onda plana en un medio unidimensional.

III-B. Extensión al caso tridimensional

En tres dimensiones, la onda plana en $\mathbf{x} = [x_1, x_2, x_3]^T$ se escribe:

$$s(\mathbf{x}, t) = s(\mathbf{0}, t)\, e^{j\beta(x_1 \cos\varphi \sin\theta + x_2 \sin\varphi \sin\theta + x_3 \cos\theta)}, \quad (11)$$

donde φ es el **azimut** y θ la **elevación**. Aquí $s(\mathbf{0}, t)$ es la señal de la fuente en su origen. En DOA lo importante no es el contenido exacto de $s(\mathbf{0}, t)$, sino cómo la propagación introduce diferencias de fase entre sensores [15].

III-C. Arreglos linealmente uniformes

Un caso clásico es el **Uniform Linear Array (ULA)**: una fila de M sensores equiespaciados (Fig. 4). Si la onda incide frontalmente, todos los micrófonos reciben la señal en fase. En cambio, con un ángulo oblicuo aparecen retardos progresivos: cuanto más inclinado, mayor la diferencia de tiempos entre sensores.

En términos físicos, cada micrófono mide la misma señal con una **fase distinta**. Ese patrón de desfases codifica la dirección de llegada. Matemáticamente, la señal en el micrófono m-ésimo es:

$$s_m(t) = s(0, t)\, a_m(\varphi, \theta), \quad (12)$$

donde $s(0, t)$ es la señal de referencia y $a_m(\varphi, \theta)$ un factor de fase:

$$a_m(\varphi, \theta) = e^{j\beta m d \cos\varphi \sin\theta}. \quad (13)$$

III-D. Steering vector

Agrupando todos los factores de fase en un vector columna se obtiene el **steering vector**:

$$\mathbf{a}(\varphi, \theta) = \begin{bmatrix} 1 \\ e^{j\beta d \cos\varphi \sin\theta} \\ e^{j2\beta d \cos\varphi \sin\theta} \\ \vdots \\ e^{j(M-1)\beta d \cos\varphi \sin\theta} \end{bmatrix}. \quad (14)$$

Este vector es la **firma espacial** del arreglo: si la onda proviene de (φ, θ), las fases se alinean, y los algoritmos de DOA (DAS, Capon, MUSIC, ESPRIT) lo utilizan como base para estimar la dirección de la fuente [15], [12].

III-E. Delay-and-Sum Beamformer (DAS)

El método más clásico y sencillo para estimar la dirección de arribo es el **beamformer de retraso y suma** (DAS, por sus siglas en inglés). La idea es intuitiva: si se supone un ángulo de llegada, se compensan los retardos con los que la onda alcanza a cada sensor, y luego se suman las señales.

- Cuando el ángulo supuesto coincide con el real, las señales quedan alineadas en fase y se refuerzan al sumarse (suma constructiva).
- Si el ángulo no corresponde, las señales llegan desfasadas y se atenúan entre sí (suma destructiva).

De esta forma, basta recorrer un conjunto de ángulos candidatos y calcular la potencia de la salida del beamformer para obtener el **espectro espacial DAS**: los máximos de este espectro indican las direcciones estimadas de llegada de la señal [15].

Características principales:

- Implementación extremadamente simple y de bajo costo computacional.
- No requiere información adicional sobre el entorno ni sobre las señales.
- Resolución angular limitada: los lóbulos principales son anchos, lo que dificulta separar fuentes cercanas.
- Rechazo a interferencias poco eficiente.
- Mejora con mayor número de sensores y mayor apertura del arreglo.

Formulación matemática: Sea $\mathbf{u}(t) \in \mathbb{C}^M$ el vector de observaciones de un arreglo de M sensores, y $R_{uu} = \mathbb{E}\{\mathbf{u}(t)\mathbf{u}^H(t)\}$ su matriz de correlación. El *steering vector* para un ángulo candidato φ es:

$$\mathbf{a}(\varphi) = \begin{bmatrix} 1 \\ e^{j2\pi d_{\text{norm}} \cos\varphi} \\ e^{j2\pi d_{\text{norm}} \cdot 2 \cos\varphi} \\ \vdots \\ e^{j2\pi d_{\text{norm}} (M-1) \cos\varphi} \end{bmatrix}, \quad (15)$$

donde $d_{\text{norm}} = d/\lambda$ es el espaciamiento normalizado entre elementos.

La salida del beamformer apuntando hacia φ es:

$$y(t) = \mathbf{a}^H(\varphi)\, \mathbf{u}(t), \quad (16)$$

y su potencia promedio:

$$P_{DAS}(\varphi) = \mathbf{a}^H(\varphi)\, R_{uu}\, \mathbf{a}(\varphi). \quad (17)$$

Al graficar $P_{DAS}(\varphi)$ en función del ángulo, los picos corresponden a las direcciones estimadas de arribo. En la Fig. 7 se muestra un ejemplo de este espectro simulado en MATLAB para un arreglo lineal.

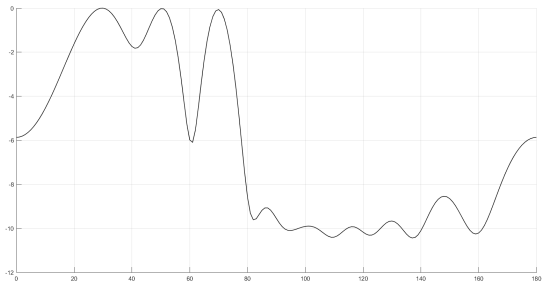


Figura 7: Espectro espacial obtenido con el beamformer Delay-and-Sum (DAS). Se simularon tres fuentes incidentes en 30°, 50° y 70°, con una relación señal-ruido (SNR) de 5 dB, usando un arreglo de 10 sensores uniformemente espaciados con $d_{norm} = 0,5$. Los picos del espectro corresponden a las direcciones estimadas de arribo.

III-F. Capon Beamformer (MVDR)

El beamformer de **Capon**, también conocido como **MVDR (Minimum Variance Distortionless Response)**, se propuso como una mejora directa del método Delay-and-Sum. Su idea central es apuntar hacia una dirección deseada sin distorsionar la señal proveniente de allí, al mismo tiempo que se minimiza la energía recibida desde todas las demás direcciones.

De manera intuitiva, si el DAS se asemeja a una linterna con un haz ancho, Capon actúa como un láser mucho más fino: concentra la ganancia únicamente en el ángulo de interés y suprime la contribución de señales interferentes [15].

- **Ganancia unitaria en la dirección deseada**: se garantiza que la señal en el ángulo φ no se distorsione.
- **Mínima potencia desde otras direcciones**: los pesos del arreglo se ajustan para cancelar la energía procedente del resto del espacio.

Características:

- Mayor resolución que DAS: lóbulos principales más estrechos y mejor discriminación angular.
- Rechazo eficaz de interferencias fuera de la dirección de interés.
- Requiere una estimación precisa de la matriz de correlación R_{uu}; con pocos datos puede degradarse.
- Más costoso computacionalmente, ya que exige invertir matrices.
- Sensible a errores de calibración en el arreglo.

Formulación matemática: El problema de diseño de Capon se plantea como una optimización:

$$\min_{\mathbf{w}} \ \mathbf{w}^H R_{uu}\, \mathbf{w} \quad \text{sujeto a} \quad \mathbf{w}^H \mathbf{a}(\varphi) = 1, \tag{18}$$

donde R_{uu} es la matriz de correlación de las señales, $\mathbf{a}(\varphi)$ es el *steering vector*, y $\mathbf{w}$ los pesos del beamformer.

La solución, obtenida con multiplicadores de Lagrange, es:

$$\mathbf{w}_c(\varphi) = \frac{R_{uu}^{-1}\, \mathbf{a}(\varphi)}{\mathbf{a}^H(\varphi)\, R_{uu}^{-1}\, \mathbf{a}(\varphi)}. \tag{19}$$

El espectro espacial asociado al beamformer de Capon es:

$$P_{\text{Capon}}(\varphi) = \frac{1}{\mathbf{a}^H(\varphi)\, R_{uu}^{-1}\, \mathbf{a}(\varphi)}. \tag{20}$$

Los máximos de $P_{\text{Capon}}(\varphi)$ señalan las direcciones estimadas de arribo, con lóbulos mucho más estrechos que los de DAS, tal como se aprecia en la Fig. 8.

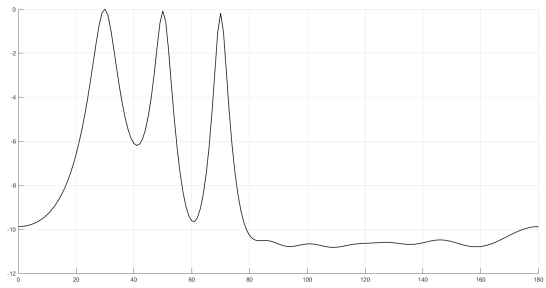


Figura 8: Espectro espacial obtenido con el beamformer de Capon (MVDR). Se simularon tres fuentes incidentes en $30°$, $50°$ y $70°$, con una relación señal-ruido (SNR) de 5 dB, usando un arreglo de 10 sensores con espaciamiento normalizado $d_{norm} = 0,5$. En comparación con DAS, los lóbulos principales son más estrechos, lo que permite una mejor discriminación angular entre fuentes cercanas.

III-G. MUSIC (Multiple Signal Classification)

El algoritmo **MUSIC** es uno de los métodos más influyentes para la estimación de dirección de arribo. Su principal fortaleza es la capacidad de alcanzar **alta resolución**, separando fuentes muy cercanas angularmente incluso con un número moderado de sensores [12].

La idea se basa en descomponer la matriz de correlación R_{uu} en dos subespacios ortogonales:

- El **espacio de señal**, generado por los *steering vectors* de las fuentes reales.
- El **espacio de ruido**, ortogonal al espacio de señal.

Un steering vector correspondiente a una dirección real de llegada es ortogonal al subespacio de ruido. Por lo tanto, al proyectar $\mathbf{a}(\varphi)$ sobre ese subespacio, la proyección se anula en los ángulos correctos.

Características:

- Alta resolución, capaz de distinguir fuentes muy próximas.
- Requiere conocer o estimar el número de fuentes presentes.
- Sensible a correlaciones entre señales y a errores de calibración del arreglo.

Formulación matemática: La descomposición en valores propios de R_{uu} permite separar:

$$R_{uu} = E_s \Lambda_s E_s^H + E_n \Lambda_n E_n^H, \tag{21}$$

donde E_s contiene los autovectores asociados a las fuentes (espacio de señal) y E_n los del ruido.

El espectro espacial de MUSIC se define como:

$$P_{\text{MUSIC}}(\varphi) = \frac{1}{\mathbf{a}^H(\varphi)\, E_n E_n^H\, \mathbf{a}(\varphi)}. \tag{22}$$

Los picos de $P_{\text{MUSIC}}(\varphi)$ corresponden a las direcciones estimadas de llegada.

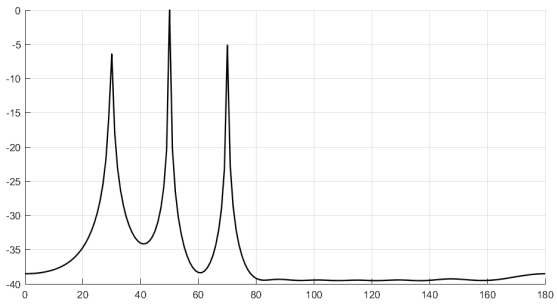


Figura 9: Espectro espacial obtenido con el algoritmo MUSIC. Se simularon tres fuentes incidentes en $30°$, $50°$ y $70°$, con SNR de 5 dB y un arreglo ULA de 10 sensores con $d_{norm} = 0,5$. Los picos estrechos permiten distinguir claramente las fuentes.

III-H. Root-MUSIC

El método **Root-MUSIC** es una variante diseñada para evitar el barrido angular. En lugar de evaluar $P_{\text{MUSIC}}(\varphi)$ punto a punto, transforma el problema en encontrar raíces de un polinomio complejo asociado al subespacio de ruido.

Ventajas:

- Reduce la complejidad computacional (no requiere barrido angular).
- Ofrece estimaciones más precisas en comparación con MUSIC clásico.

Limitación principal:

- Está restringido a arreglos lineales uniformes (ULA).

Formulación matemática: Se define el polinomio:

$$Q[z] = \sum_{k=-(M-1)}^{M-1} q_k z^{-k}, \qquad q_k = \sum_{m-l=k} C(m,l), \tag{23}$$

con $z = e^{j2\pi \frac{d}{\lambda} \cos\varphi}$ y $C = E_n E_n^H$.

Las raíces z_i cercanas al círculo unitario determinan los ángulos estimados:

$$\hat{\varphi}_i = \cos^{-1}\left(\frac{\angle z_i}{2\pi d}\right). \tag{24}$$

III-I. Spatial Smoothing MUSIC

El **spatial smoothing** es una extensión de MUSIC para manejar señales *coherentes* (por ejemplo, en presencia de multitrayectoria). Cuando las señales son coherentes, la matriz R_{uu} pierde rango y MUSIC clásico falla. El smoothing restaura el rango promediando matrices de correlación de subarreglos [12].

Características:

- Permite detectar señales coherentes que MUSIC clásico no puede resolver.
- A costa de reducir la apertura efectiva del arreglo (menor resolución).
- *Forward smoothing*: detecta hasta $M/2$ señales coherentes.
- *Forward/Backward smoothing*: hasta $2M/3$ señales coherentes.

Formulación matemática: Sea $K = M - p + 1$ el número de subarreglos forward posibles. La correlación promedio es:

$$R^f = \frac{1}{K} \sum_{k=0}^{K-1} R_k^f. \tag{25}$$

Si además se consideran subarreglos backward, se obtiene:

$$R = \frac{R^f + R^b}{2}. \tag{26}$$

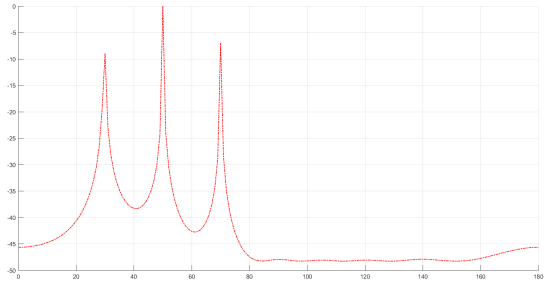


Figura 10: Ejemplo de MUSIC con spatial smoothing (SS-MUSIC). Se consideran tres fuentes coherentes en $30°$, $50°$ y $70°$, con SNR de 5 dB y 10 sensores ($d_{norm} = 0,5$). El smoothing restaura el rango de la matriz de correlación y permite separar correctamente las fuentes.

III-J. ESPRIT (Estimation of Signal Parameters via Rotational Invariance Techniques)

El algoritmo **ESPRIT** es una alternativa a MUSIC que evita el barrido angular. En lugar de calcular un espectro y buscar picos, se apoya en la **invariancia rotacional** entre subarreglos de un ULA (Uniform Linear Array). Esto permite estimar directamente las direcciones de arribo resolviendo un problema algebraico [12].

Idea general:

- Se divide el arreglo de M sensores en dos subarreglos idénticos, desplazados por una distancia fija Δx.
- La señal en el segundo subarreglo es equivalente a la del primero, multiplicada por un factor de fase dependiente del ángulo de arribo.
- Esta relación se modela como una **invariancia rotacional**, lo que permite estimar los ángulos a partir de los autovalores de una matriz reducida.

Mientras que MUSIC "escanea" ángulo por ángulo para detectar picos, ESPRIT utiliza propiedades algebraicas del arreglo y evita la búsqueda explícita.

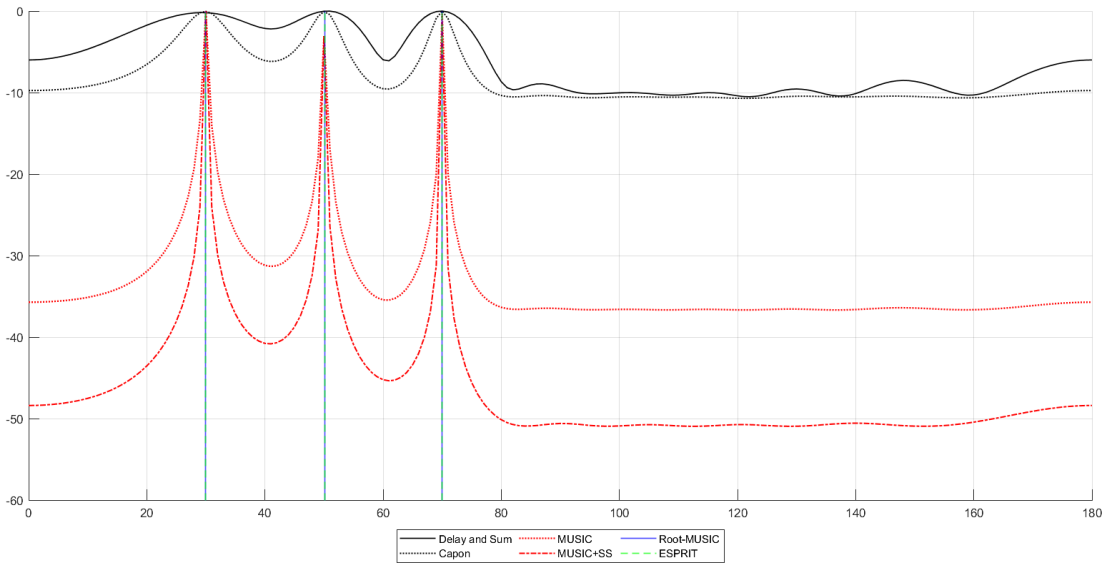


Figura 11: Comparación de métodos de estimación de DOA: DAS muestra lóbulos anchos y baja resolución; Capon mejora la nitidez pero aún con interferencias; MUSIC separa claramente las fuentes en condiciones adversas; y ESPRIT obtiene los ángulos sin necesidad de barrido.

Características:

- No requiere barrido angular, reduciendo la complejidad frente a MUSIC.
- Menor tiempo de cálculo y menos sensible a discretización de ángulos.
- Exige que el arreglo posea subestructuras desplazadas (ej. ULA).
- Sensible a errores de calibración o desviaciones geométricas.

Formulación matemática: Dividiendo el arreglo en dos subarreglos, las señales medidas se modelan como:

$$\mathbf{u}_0(t) = A\,\mathbf{s}(t) + \mathbf{n}_0(t), \qquad \mathbf{u}_1(t) = A\Phi\mathbf{s}(t) + \mathbf{n}_1(t), \quad (27)$$

donde A es la matriz de steering del primer subarreglo, y Φ una matriz diagonal de factores de fase:

$$\Phi = \operatorname{diag}\left(e^{j\gamma_1}, e^{j\gamma_2}, \ldots, e^{j\gamma_L}\right), \qquad \gamma_l = \frac{2\pi}{\lambda}\,\Delta x \cos\varphi_l.$$

A partir del subespacio de señal V_s, se extraen dos bloques V_0 y V_1 correspondientes a cada subarreglo. La relación de invariancia se expresa como:

$$V_1 \approx V_0\Psi, \quad (28)$$

donde Ψ es una matriz de rotación que contiene la información angular.

Los autovalores de Ψ son:

$$\exp(j\gamma_l) = e^{j\frac{2\pi}{\lambda}\,\Delta x \cos\varphi_l},$$

a partir de los cuales se obtiene directamente la estimación de los ángulos:

$$\hat{\varphi}_l = \cos^{-1}\left(\frac{\lambda}{2\pi\Delta x}\,\angle(e^{j\gamma_l})\right), \quad l = 0, \ldots, L-1. \quad (29)$$

Finalmente, para contrastar los distintos métodos se simuló un escenario común (Fig. 11) con tres fuentes en $30°$, $50°$ y $70°$, relación señal-ruido de 5 dB, un arreglo ULA de 10 sensores con espaciamiento normalizado $d_{norm} = 0,5$, y multitrayectoria (trayectoria directa más una retardada). En el caso de MUSIC con *spatial smoothing*, se emplearon $M-2$ subarreglos solapados.

IV. Deep Learning aplicado a audio

IV-A. Contextos matemáticos y preprocesamiento

El análisis de señales acústicas mediante *Deep Learning* requiere transformar el audio crudo en representaciones que capturen adecuadamente sus propiedades espectrales y temporales. Para ello se emplean herramientas de la teoría de Fourier y representaciones perceptuales que aproximan el comportamiento del sistema auditivo humano. En lo que sigue se presentan las bases matemáticas de tres transformaciones fundamentales: la Transformada Rápida de Fourier (FFT), la Transformada de Fourier de Tiempo Corto (STFT) y la escala Mel.

IV-A1. Transformada Rápida de Fourier (FFT): Dada una señal discreta $x[n]$ de longitud N, su Transformada Discreta de Fourier (DFT) está definida como

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j\frac{2\pi}{N}kn}, \quad k = 0, 1, \ldots, N-1. \tag{30}$$

La DFT descompone la señal en senoidales complejas ortogonales, revelando la distribución de energía en frecuencia [2], [3]. Su cálculo directo tiene costo $\mathcal{O}(N^2)$, pero el algoritmo FFT lo reduce a $\mathcal{O}(N \log N)$, habilitando aplicaciones en tiempo real como detección acústica.

Es importante notar que la DFT brinda una descripción global: indica magnitud y fase de cada componente frecuencial, pero pierde información sobre el instante en que ocurrió. En bioacústica esto es relevante: el zumbido de un mosquito puede variar en fracciones de segundo, lo que motiva el uso de representaciones con resolución conjunta en tiempo y frecuencia.

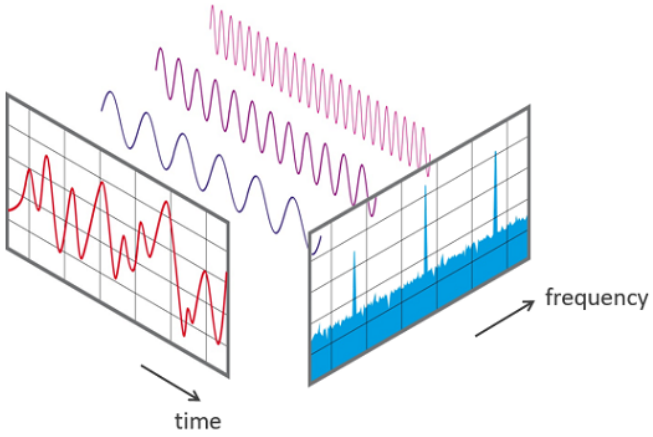


Figura 12: Conversión de una señal temporal a su representación espectral mediante FFT.

IV-A2. Transformada de Fourier de Tiempo Corto (STFT): La STFT extiende la DFT incorporando localización temporal. La señal se multiplica por una ventana $w[n]$ de duración finita, que selecciona un segmento, y se desplaza en el tiempo mediante un parámetro m. Su definición es:

$$X(m, \omega) = \sum_{n=-\infty}^{\infty} x[n]\, w[n-m]\, e^{-j\omega n}. \tag{31}$$

El resultado es una función bidimensional que describe cómo varía el contenido espectral en el tiempo, dando lugar al *espectrograma*. La resolución temporal y frecuencial dependen inversamente de la longitud de la ventana: ventanas cortas capturan sucesos breves pero con poca resolución espectral, mientras que ventanas largas ofrecen buena resolución en frecuencia pero pierden detalle temporal. Esta dualidad, conocida como principio de incertidumbre tiempo–frecuencia, condiciona el diseño del preprocesamiento [14].

En detección de mosquitos, la STFT permite observar la evolución dinámica del zumbido, que puede variar en frecuencia fundamental por el movimiento del insecto o por interferencias ambientales.

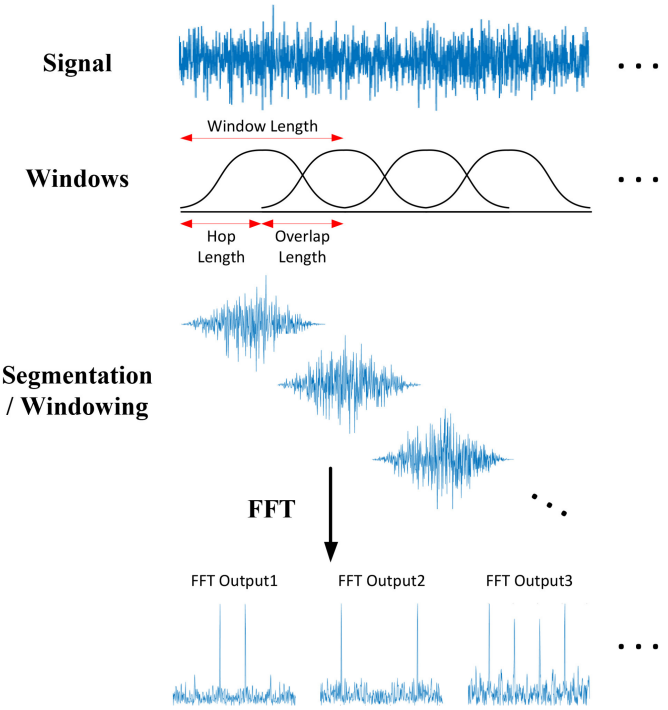


Figura 13: Esquema de la STFT mostrando segmentación por ventanas.

IV-A3. Escala Mel: Aunque la STFT proporciona un espectro en Hz, el sistema auditivo humano no percibe las frecuencias de manera lineal. Somos más sensibles a bajas frecuencias: diferenciar entre 100 y 200 Hz resulta más fácil que entre 5000 y 5100 Hz. Para aproximar esta percepción se usa la escala Mel, que relaciona la frecuencia física f con una perceptual f_{mel}:

$$f_{\text{mel}} = 2595 \log_{10}\left(1 + \frac{f}{700}\right). \tag{32}$$

El cálculo de un *mel-spectrograma* consiste en aplicar un banco de filtros triangulares, distribuidos equidistantemente en la escala Mel, sobre el espectrograma lineal. La energía espectral se proyecta a un dominio perceptual comprimido, reduciendo dimensionalidad e introduciendo cierta invariancia frente a variaciones irrelevantes para el oído humano.

En *Deep Learning* aplicado a audio, el mel-spectrograma es una de las representaciones más utilizadas como entrada a CNNs, pues concentra la información discriminativa y mejora la robustez frente a ruido ambiental [9], [1].

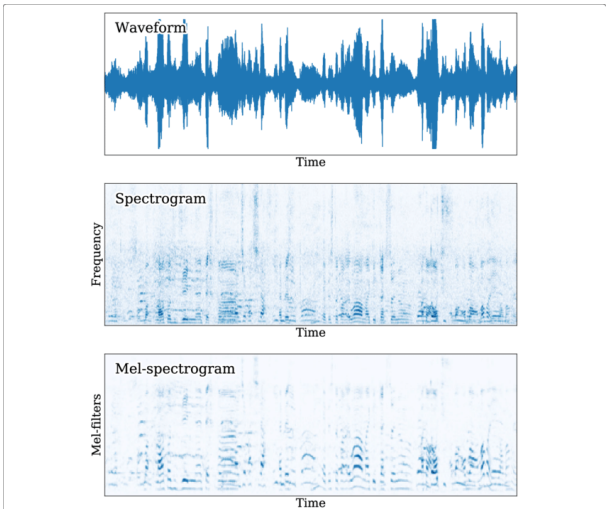


Figura 14: Waveform, espectrograma y mel-spectrograma de un segmento de audio de 10 segundos (Google AudioSet). El mel-spectrograma, basado en la escala perceptual Mel, ofrece mayor resolución en bajas frecuencias que el espectrograma lineal.

IV-B. CNNs aplicadas a HumBugDB

El conjunto de datos *HumBugDB* constituye un recurso extenso y representativo de grabaciones acústicas de mosquitos en entornos de laboratorio y de campo [1]. La señal característica del mosquito es un zumbido cuasi-periódico producido por el batido de alas, con frecuencia fundamental y armónicos. Su detección en grabaciones reales es desafiante, ya que suele estar enmascarada por ruidos ambientales como viento, tráfico o voces humanas.

Inferencia bayesiana mediante MC Dropout: En redes profundas, las predicciones suelen ser tratadas como determinísticas, sin cuantificar la confianza. Gal y Ghahramani (2016) demostraron que el *dropout* —originalmente introducido como técnica de regularización— puede interpretarse como una aproximación bayesiana cuando se mantiene activo también durante la inferencia.

Este procedimiento, conocido como *Monte Carlo Dropout* (MC Dropout), consiste en realizar múltiples pasadas de inferencia con distintas máscaras de dropout, generando una distribución de salidas en lugar de un valor único. A partir de esa distribución se obtienen medidas de incertidumbre, como la varianza predictiva, lo que permite distinguir entre predicciones seguras y dudosas [10]. En audio bioacústico esta capacidad resulta esencial, pues las señales suelen estar contaminadas por ruido.

IV-B1. Motivación del uso de CNNs: Las Redes Neuronales Convolucionales (CNNs) son especialmente adecuadas porque trabajan sobre representaciones bidimensionales como los mel-spectrogramas. Una capa convolucional aplica filtros $w^{(k)}$ sobre la entrada x de acuerdo con

$$y_{i,j}^{(k)} = \sigma\left(\sum_{m=0}^{M-1}\sum_{n=0}^{N-1} w_{m,n}^{(k)}\, x_{i+m,\,j+n} + b^{(k)}\right), \qquad (33)$$

donde $b^{(k)}$ es el sesgo y $\sigma(\cdot)$ la activación.

Este mecanismo permite aprender filtros jerárquicos: los primeros sensibles a patrones locales (armónicos, periodicidades) y los posteriores a representaciones más abstractas que diferencian especies o condiciones de grabación.

IV-B2. Aplicación y robustez en HumBugDB: En HumBugDB, las CNNs se entrenan sobre ventanas de audio convertidas en mel-spectrogramas, que concentran la estructura espectral relevante en un dominio perceptual comprimido. Campos et al. (2022) mostraron que, incorporando *data augmentation* acústico —mezcla con ruido de fondo, variaciones de tono y estiramiento temporal—, los modelos aprenden representaciones invariantes a perturbaciones comunes, mejorando su desempeño en ambientes ruidosos [9].

Los *kernels* se optimizan mediante retropropagación y no se fijan como filtros predefinidos, lo que permite a la red adaptarse a patrones específicos de mosquitos. La normalización por lotes y el *dropout* introducen estocasticidad que previene el sobreajuste. Además, al mantener dropout activo en inferencia, se generan múltiples predicciones por entrada, lo que habilita la estimación de incertidumbre asociada a cada salida [10].

IV-B3. Resultados: Los estudios iniciales reportaron que las CNNs superan ampliamente a clasificadores tradicionales (SVMs, Random Forests) cuando se entrenan con mel-spectrogramas de HumBugDB. Además de detectar la presencia de mosquitos, los modelos pueden diferenciar especies, lo cual aporta valor epidemiológico para programas de control de vectores [1], [9]. La combinación de *data augmentation*, kernels entrenables y regularización bayesiana mediante dropout convierte a estas arquitecturas en una herramienta robusta para la detección acústica en condiciones reales.

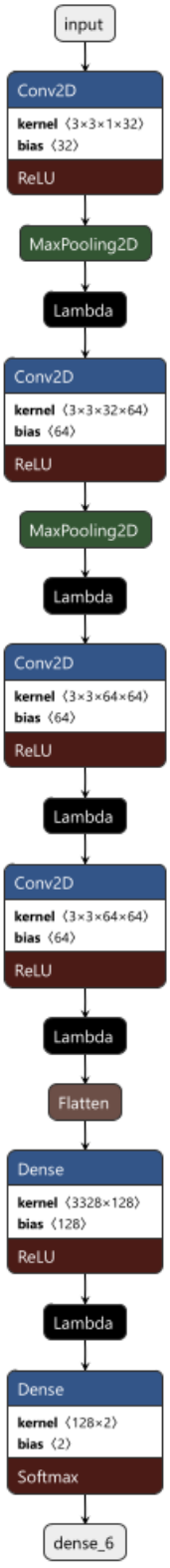


Figura 15: Arquitectura típica de una CNN utilizada en HumBugDB. El pipeline incluye: entrada de mel-spectrogramas, convoluciones con activación ReLU y max-pooling, aplanamiento de características y capas densas para la clasificación. Manteniendo dropout activo también en la fase de inferencia es posible estimar la incertidumbre asociada a cada predicción.

IV-C. Modelos secuenciales

RNN y LSTM: Otra familia de arquitecturas fundamentales en audio son las *Redes Neuronales Recurrentes* (RNNs), diseñadas para procesar secuencias temporales. En una RNN, la salida en cada instante t depende no solo de la entrada x_t, sino también del estado oculto h_{t-1}, lo que permite modelar dependencias a lo largo del tiempo.

El entrenamiento de RNNs estándar enfrenta el problema del *gradiente desvanecido o explosivo*. Durante la retropropagación a través del tiempo (BPTT), los gradientes se multiplican repetidamente por derivadas de funciones de activación. Si estas son menores que uno, los gradientes tienden a cero (*vanishing*); si son mayores que uno, crecen exponencialmente (*exploding*). Como consecuencia, la red tiene dificultades para aprender dependencias de largo plazo de manera estable.

Para mitigar este problema se introdujeron variantes como las *Long Short-Term Memory* (LSTM) y las *Gated Recurrent Units* (GRU), que incorporan compuertas de control del flujo de información:

- La **puerta de olvido** decide qué parte del estado anterior se descarta.
- La **puerta de entrada** regula qué información nueva se incorpora a la memoria.
- La **puerta de salida** determina qué fracción del estado interno influye en la salida actual.

Gracias a este diseño, las LSTM mantienen gradientes estables en secuencias largas, permitiendo capturar dependencias temporales de distinto alcance [16]. Estas arquitecturas han sido ampliamente utilizadas en tareas de *Music Information Retrieval* (MIR), como clasificación de género musical, transcripción automática o separación de fuentes, y continúan siendo un estándar en el modelado de series temporales [16].

IV-D. Deep Learning para dirección de arribo y beamforming

La estimación de dirección de arribo (DoA) es un problema central en radar, comunicaciones inalámbricas y procesamiento acústico. Los algoritmos clásicos como MUSIC destacan por su alta resolución angular, pero presentan limitaciones conocidas: baja robustez con pocas muestras (*snapshots*), sensibilidad al ruido y dificultad para localizar fuentes coherentes. Estas restricciones reducen su eficacia en escenarios prácticos.

Un enfoque reciente, denominado **DLMUSIC**, combina técnicas de *Deep Learning* con el marco clásico de MUSIC [18]. La propuesta introduce dos innovaciones principales:

- **Estimación de la covarianza:** en lugar de usar la matriz de covarianza empírica, se entrenan redes convolucionales (CNNs) para predecir una versión más estable y robusta, menos sensible al ruido y al número de snapshots.
- **Espectro objetivo suavizado:** en vez de aproximar el espectro MUSIC real (que puede presentar picos distorsionados y lóbulos espurios en baja SNR), se utiliza un *espectro idealizado* compuesto por funciones Gaussianas centradas en las direcciones verdaderas de llegada. Esta representación

suavizada actúa como objetivo de entrenamiento, guiando a la red hacia picos definidos y reduciendo falsos máximos.

Tras la estimación de la covarianza por las CNNs, el algoritmo procede con la descomposición en autovalores y la separación de subespacios propia de MUSIC, preservando así la interpretabilidad del método clásico. De esta manera, DLMUSIC no reemplaza la teoría de subespacios, sino que corrige las etapas más frágiles mediante aprendizaje profundo.

De acuerdo con los resultados reportados, este enfoque supera al MUSIC clásico en escenarios de **baja SNR**, con **pocos snapshots** y en presencia de **fuentes coherentes** [18]. Ello demuestra cómo el *Deep Learning* puede complementar a los algoritmos tradicionales, reforzando su desempeño en condiciones adversas y acercándolos a un uso más confiable en aplicaciones modernas.

V. Implementación en hardware embebido (FPGA)

El despliegue de algoritmos de *Deep Learning* en sistemas embebidos requiere procesar datos en tiempo real bajo restricciones de consumo energético y tamaño. Estas condiciones no siempre pueden cumplirse con plataformas de propósito general como CPU o GPU. En este contexto, las **FPGA** (Field Programmable Gate Arrays) constituyen una alternativa atractiva, al permitir arquitecturas paralelas y personalizadas con menor consumo y latencias más predecibles, lo que las hace idóneas para aplicaciones portátiles e integradas [17].

De acuerdo con la revisión de Shawahna et al., las FPGA se emplean principalmente como **aceleradores de inferencia**, explotando:

- **Optimización de precisión**: uso de aritmética en punto fijo o de baja resolución.
- **Paralelismo jerárquico**: a nivel de datos, filtros y capas.
- **Compresión y reutilización de pesos**: reduce accesos a memoria externa, principal cuello de botella.
- **Pipeline de capas**: la salida de cada etapa alimenta de inmediato a la siguiente, evitando almacenamientos intermedios costosos.

No obstante, la implementación de redes neuronales en FPGA presenta **desafíos relevantes**: mayores tiempos de desarrollo frente a CPU/GPU, limitación de recursos lógicos (celdas, DSPs, BRAM) y compatibilidad aún parcial con frameworks de *Deep Learning*. Aun así, la literatura destaca que las FPGA se perfilan como una de las soluciones más prometedoras para llevar inteligencia artificial a sistemas embebidos de tamaño reducido [17].

VI. Conclusiones

Este trabajo recorrió un arco que va desde técnicas clásicas como los *Phase-Locked Loops* (PLL) y *Matched Filters*, pasando por métodos de estimación de dirección de arribo (DAS, Capon, MUSIC, ESPRIT), hasta la incorporación de *Deep Learning* y su factibilidad en *hardware* embebido.

La comparación mostró el aporte de cada enfoque:

- Los métodos clásicos ofrecen interpretabilidad, estabilidad matemática y bajo costo computacional.
- Los métodos modernos basados en *Deep Learning* aportan robustez ante ruido y variabilidad de entorno.
- La estimación de DOA agrega valor espacial frente a la mera detección espectral.

Un resultado clave es que la **combinación** de enfoques clásicos y modernos abre un **paradigma híbrido** prometedor. Ejemplos de ello son el uso de modelos de *Deep Learning* para estabilizar etapas sensibles (como la estimación de covarianza) manteniendo la inferencia angular mediante teoría de subespacios (MUSIC/ESPRIT), o bien la aplicación de preprocesamiento clásico (filtros, *matched filtering*, *mel*-espectrogramas) como insumo para clasificadores modernos. Esta sinergia mejora resolución y robustez, al tiempo que conserva trazabilidad en el flujo de señal.

Finalmente, las **FPGA** emergen como habilitador práctico para llevar estos esquemas híbridos a tiempo real en sistemas portátiles de bajo consumo. Persisten desafíos —tiempos de desarrollo, limitación de recursos lógicos y ecosistema aún parcial de herramientas—, pero la ruta DL + DOA + FPGA perfila una plataforma sólida para implementaciones embebidas y futuras líneas de investigación.

Referencias

[1] I. Kiskin, M. Sinka, A. D. Cobb, W. Rafique, L. Wang, D. Zilli, B. Gutteridge, R. Dam, T. Marinos, Y. Li, D. Msaky, E. Kaindoa, G. Killeen, E. Herreros-Moya, K. Willis, and S. J. Roberts, "HumBugDB: A large-scale acoustic mosquito dataset," in *Proc. NeurIPS Datasets and Benchmarks*, 2021. [Online]. Available: https://doi.org/10.5281/zenodo.4904800

[2] S. Haykin, *Communication Systems*, 4th ed. John Wiley & Sons, 2001.

[3] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. McGraw-Hill Higher Education, 2007.

[4] I. Collins, "Phase-Locked Loop (PLL) Fundamentals," *Analog Dialogue*, vol. 52, no. 3, Jul. 2018. Analog Devices. [Online]. Available: https://www.analog.com/media/en/analog-dialogue/volume-52/number-3/phase-locked-loop-pll-fundamentals.pdf

[5] Analog Devices, "MT-086: Fundamentals of Phase Locked Loops (PLLs)," Tutorial Note, 2009. [Online]. Available: https://www.analog.com/media/en/training-seminars/tutorials/mt-086.pdf

[6] G. Nash, "Phase-Locked Loop Design Fundamentals," *Application Note AN535*, Freescale/NXP, Rev. 1.0, 1994. [Online]. Available: https://www.nxp.com/docs/en/application-note/AN535.pdf

[7] Analog Devices, "System-Level Local Oscillator Phase Noise Models for Phased Arrays with Distributed PLLs," *Embedded Computing Design*, 2020. [Online]. Available: https://embeddedcomputing.com/technology/debug-and-test/system-level-lo-phase-noise-model-for-phased-arrays-with-distributed-phase-locked-loops

[8] R. Gallager, *Principles of Digital Communications*, 1st ed. Cambridge University Press, 2008.

[9] S. Campos, D. Khandelwal, S. C. Nagaraj, F. Nugen, and A. Todeschini, "Deep learning-based acoustic mosquito detection in noisy conditions using trainable kernels and augmentations," in *Proc. 30th ACM Int. Conf. Multimedia (MM)*, Lisbon, Portugal, Oct. 2022. doi: 10.1145/3503161.3551586.

[10] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning," *arXiv:1506.02142*, 2016. [Online]. Available: https://arxiv.org/abs/1506.02142

[11] Y. Han, M. Luo, X. Zhao, J. M. Guerrero, and L. Xu, "Comparative Performance Evaluation of Orthogonal-Signal-Generators based Single-Phase PLL Algorithms—A Survey," *IEEE Trans. Power Electron.*, 2015, doi: 10.1109/TPEL.2015.2466631.

[12] P. Gupta and S. P. Kar, "MUSIC and Improved MUSIC algorithm to Estimate Direction of Arrival," in *Proc. IEEE Int. Conf. on Communications and Signal Processing (ICCSP)*, 2015.

[13] A. García-García, "Análisis del uso de algoritmos de Deep Learning en un Sistema acústico de detección de larvas de insectos xilófagos," TFG, Universidad de Valladolid, Jul. 2024.
[14] J. C. Bancroft, "Introduction to matched filters," *CREWES Research Report*, vol. 14, 2002.
[15] W. Y. Yang, *MATLAB/Simulink for Digital Signal Processing*. CRC Press.
[16] L. Moysis, L. A. Iliadis, S. P. Sotiroudis, A. D. Boursianis, M. S. Papadopoulou, K.-I. D. Kokkinidis, C. Volos, P. Sarigiannidis, S. Nikolaidis, and S. K. Goudos, "Music Deep Learning: Deep Learning Methods for Music Signal Processing—A Review of the State-of-the-Art," *IEEE Access*, 2023.
[17] A. Shawahna, S. M. Sait, and A. El-Maleh, "FPGA-Based Accelerators of Deep Learning Networks for Learning and Classification: A Review," *IEEE Access*, vol. 7, pp. 7823–7859, 2019, doi: 10.1109/ACCESS.2018.2890150.
[18] Y. Bai and Z. Chen, "DLMUSIC: The Improved MUSIC Algorithm Based on Deep Learning," in *Proc. 2023 4th Int. Conf. on Computer Engineering and Application (ICCEA)*, Hangzhou, China, Apr. 7–9, 2023, doi: 10.1109/ICCEA58433.2023.10135326.
[19] Python For Everybody, "Beamforming & DOA estimation in Python," YouTube, Dec. 16, 2024. [Online]. Available: https://www.youtube.com/watch?v=hqQM6YSV5gw&t=1653s
[20] Python For Everybody, "Python implementation of Multi Target angle discrimination using MUSIC," YouTube, Feb. 20, 2025. [Online]. Available: https://www.youtube.com/watch?v=d4pYlg4YRn8
[21] Python For Everybody, "Python implementation of Multiple Signal Classification(MUSIC) Algo for Angle estimitation," YouTube, Feb. 12, 2025. [Online]. Available: https://www.youtube.com/watch?v=gJDcOqfnezk&t=1465s
[22] C. Yang, "DoA estimation methods: DAS, Capon, MUSIC, ESPRIT with MATLAB," YouTube, Apr. 26, 2025. [Online]. Available: https://www.youtube.com/watch?v=lizvrtv69lo&t=1306s